```
// ============ 文件：server/server.js ============
/**
 * 学习打卡管理平台 - 服务入口
 * 启动 HTTP 服务并挂载路由与中间件链。
 */
const express = require('express');
const path = require('path');
const config = require('./config');
const logger = require('./utils/logger');
const db = require('./db');
const routes = require('./routes');

const app = express();

app.disable('x-powered-by');

// 请求体解析
app.use(express.json({ limit: '10mb' }));
app.use(express.urlencoded({ extended: true, limit: '10mb' }));

// 统一请求日志
app.use((req, res, next) =&gt; {
  const start = Date.now();
  res.on('finish', () =&gt; {
    logger.info(`${req.method} ${req.originalUrl} ${res.statusCode} ${Date.now() - start}ms`);
  });
  next();
});

// 健康检查
app.get('/health', (req, res) =&gt; {
  res.json({ status: 'up', time: new Date().toISOString() });
});

// 业务路由
app.use('/api', routes);

// 静态资源
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// 404 兜底
app.use((req, res) =&gt; {
  res.status(404).json({ code: 404, message: '接口不存在', data: null });
});

// 全局异常兜底
app.use((err, req, res, next) =&gt; {
  logger.error('unhandled error:', err);
  res.status(500).json({ code: 500, message: '服务器内部错误', data: null });
});
```

```
async function bootstrap() {
  try {
    await db.ping();
    logger.info('数据库连接成功');
  } catch (e) {
    logger.error('数据库连接失败:', e.message);
    process.exit(1);
  }
  app.listen(config.port, config.host, () =&gt; {
    logger.info(`服务已启动: http://${config.host}:${config.port} (${config.env})`);
  });
}

bootstrap();

module.exports = app;
// ============ 文件：server/routes/admin.js ============
/**
 * 管理后台路由模块
 * 提供用户管理、任务统计与基础数据维护接口。
 */
const express = require('express');
const db = require('../db');
const { ok, fail } = require('../utils/response');
const { authRequired, adminOnly } = require('../middleware/auth');

const router = express.Router();
router.use(authRequired, adminOnly);

/**
 * 用户列表（分页 + 关键字搜索）
 * GET /api/admin/user/list
 */
router.get('/user/list', async (req, res) =&gt; {
  try {
    const pageNo = Math.max(parseInt(req.query.pageNo, 10) || 1, 1);
    const pageSize = Math.min(Math.max(parseInt(req.query.pageSize, 10) || 20, 1), 100);
    const keyword = req.query.keyword;
    const params = [];
    let where = '';
    if (keyword) {
      where = 'WHERE name LIKE ? OR account LIKE ?';
      params.push(`%${keyword}%`, `%${keyword}%`);
    }
    const totalRows = await db.query(`SELECT COUNT(*) AS total FROM t_user ${where}`, params);
    const offset = (pageNo - 1) * pageSize;
    const list = await db.query(
      `SELECT id, account, name, role, status, created_at FROM t_user ${where} ORDER BY created_at DESC
LIMIT ? OFFSET ?`,
      [...params, pageSize, offset]
```

```
    );
    return res.json(ok({ list, total: totalRows[0].total, pageNo, pageSize }));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 启用 / 禁用用户
 * POST /api/admin/user/status
 */
router.post('/user/status', async (req, res) =&gt; {
  try {
    const { userId, status } = req.body || {};
    if (!userId || ![0, 1].includes(status)) return res.json(fail('参数非法'));
    await db.query('UPDATE t_user SET status = ? WHERE id = ?', [status, userId]);
    return res.json(ok(null, '操作成功'));
  } catch (e) {
    return res.json(fail('操作失败', 500));
  }
});

/**
 * 任务统计概览
 * GET /api/admin/stats
 */
router.get('/stats', async (req, res) =&gt; {
  try {
    const [taskTotal, checkinTotal, userTotal, doneTotal] = await Promise.all([
      db.query('SELECT COUNT(*) AS c FROM t_task'),
      db.query('SELECT COUNT(*) AS c FROM t_checkin'),
      db.query('SELECT COUNT(*) AS c FROM t_user'),
      db.query("SELECT COUNT(*) AS c FROM t_task WHERE status = 'done'")
    ]);
    return res.json(ok({
      taskTotal: taskTotal[0].c,
      checkinTotal: checkinTotal[0].c,
      userTotal: userTotal[0].c,
      doneTaskTotal: doneTotal[0].c
    }));
  } catch (e) {
    return res.json(fail('统计失败', 500));
  }
});

module.exports = router;
// ============ 文件：server/routes/auth.js ============
/**
 * 认证路由模块
 * 提供注册、登录、登出与令牌刷新能力。
```

```
 */
const express = require('express');
const bcrypt = require('bcryptjs');
const db = require('../db');
const jwt = require('../utils/jwt');
const { ok, fail } = require('../utils/response');
const { requiredString } = require('../utils/validate');
const { rateLimit } = require('../middleware/rateLimit');
const { authRequired } = require('../middleware/auth');

const router = express.Router();

/**
 * 注册新账号
 * POST /api/auth/register
 */
router.post('/register', rateLimit((req) =&gt; req.ip, { windowMs: 60 * 1000, max: 10 }), async (req, res)
=&gt; {
  try {
    const { account, password, name, role } = req.body || {};
    const err = requiredString(account, '账号') || requiredString(password, '密码') || requiredString(name,
'姓名');
    if (err) return res.json(err);

    const rows = await db.query('SELECT id FROM t_user WHERE account = ?', [account]);
    if (rows.length &gt; 0) {
      return res.json(fail('账号已存在'));
    }

    const hash = await bcrypt.hash(password, 10);
    const roleValue = ['admin', 'teacher', 'parent', 'student'].includes(role) ? role : 'student';
    const result = await db.query(
      'INSERT INTO t_user (account, password, name, role, status, created_at) VALUES (?, ?, ?, ?, 1,
NOW())',
      [account, hash, name, roleValue]
    );

    const token = jwt.sign({ uid: result.insertId, role: roleValue });
    return res.json(ok({ token, userId: result.insertId }));
  } catch (e) {
    return res.json(fail('注册失败', 500));
  }
});

/**
 * 账号登录
 * POST /api/auth/login
 */
router.post('/login', rateLimit((req) =&gt; req.ip, { windowMs: 15 * 60 * 1000, max: 20 }), async (req,
res) =&gt; {
  try {
    const { account, password } = req.body || {};
    const err = requiredString(account, '账号') || requiredString(password, '密码');
    if (err) return res.json(err);
```

```
    const rows = await db.query('SELECT id, account, password, name, role, status FROM t_user WHERE
account = ?', [account]);
    if (rows.length === 0) {
      return res.json(fail('账号或密码错误', 401));
    }
    const user = rows[0];
    const matched = await bcrypt.compare(password, user.password);
    if (!matched) {
      return res.json(fail('账号或密码错误', 401));
    }
    if (user.status !== 1) {
      return res.json(fail('账号已被禁用，请联系管理员', 403));
    }

    const token = jwt.sign({ uid: user.id, role: user.role });
    return res.json(ok({ token, userId: user.id, name: user.name, role: user.role }));
  } catch (e) {
    return res.json(fail('登录失败', 500));
  }
});

/**
 * 获取当前登录用户信息
 * GET /api/auth/me
 */
router.get('/me', authRequired, (req, res) =&gt; {
  return res.json(ok(req.user));
});

/**
 * 登出（由前端丢弃令牌）
 * POST /api/auth/logout
 */
router.post('/logout', authRequired, (req, res) =&gt; {
  return res.json(ok(null, '已退出登录'));
});

module.exports = router;
// ============ 文件：server/routes/badge.js ============
/**
 * 徽章路由模块
 * 提供成就徽章解锁查询与解锁记录写入。
 */
const express = require('express');
const db = require('../db');
const { ok, fail } = require('../utils/response');
const { authRequired } = require('../middleware/auth');

const router = express.Router();
```

```
const BADGE_LIST = [
  { key: 'first_checkin', name: '初次打卡', desc: '完成第一次打卡', icon: 'badge_01' },
  { key: 'seven_days', name: '坚持一周', desc: '连续打卡 7 天', icon: 'badge_02' },
  { key: 'month_master', name: '月度达人', desc: '当月打卡 20 次', icon: 'badge_03' },
  { key: 'task_complete', name: '任务先锋', desc: '累计完成任务 10 个', icon: 'badge_04' },
  { key: 'level_up', name: '等级成长', desc: '达到等级 5', icon: 'badge_05' }
];

/**
 * 查询我的徽章墙
 * GET /api/badge/mine
 */
router.get('/mine', authRequired, async (req, res) =&gt; {
  try {
    const rows = await db.query(
      'SELECT badge_key, unlocked_at FROM t_badge_unlock WHERE student_id = ?',
      [req.user.id]
    );
    const unlocked = {};
    rows.forEach((r) =&gt; {
      unlocked[r.badge_key] = r.unlocked_at;
    });
    const list = BADGE_LIST.map((b) =&gt; ({
      ...b,
      unlocked: Boolean(unlocked[b.key]),
      unlockedAt: unlocked[b.key] || null
    }));
    return res.json(ok(list));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 后台徽章定义列表
 * GET /api/badge/defs
 */
router.get('/defs', authRequired, (req, res) =&gt; {
  return res.json(ok(BADGE_LIST));
});

module.exports = router;
// ============ 文件：server/routes/checkin.js ============
/**
 * 打卡路由模块
 * 提供任务打卡的提交、查询与审核管理。
 */
const express = require('express');
const db = require('../db');
const { ok, fail } = require('../utils/response');
```

```
const { authRequired } = require('../middleware/auth');
const checkinService = require('../services/checkinService');

const router = express.Router();

/**
 * 提交打卡
 * POST /api/checkin/create
 */
router.post('/create', authRequired, async (req, res) =&gt; {
  try {
    const body = req.body || {};
    if (!body.taskId) return res.json(fail('任务 ID 不能为空'));

    const checkinId = await checkinService.createCheckin(body, req.user);
    return res.json(ok({ checkinId }, '打卡成功'));
  } catch (e) {
    return res.json(fail('打卡提交失败', 500));
  }
});

/**
 * 打卡列表（按任务）
 * GET /api/checkin/list
 */
router.get('/list', authRequired, async (req, res) =&gt; {
  try {
    const taskId = req.query.taskId;
    const studentId = req.query.studentId;
    const status = req.query.status;
    const conds = [];
    const params = [];
    if (taskId) {
      conds.push('task_id = ?');
      params.push(taskId);
    }
    if (studentId) {
      conds.push('student_id = ?');
      params.push(studentId);
    }
    if (status) {
      conds.push('audit_status = ?');
      params.push(status);
    }
    const where = conds.length &gt; 0 ? `WHERE ${conds.join(' AND ')}` : '';
    const rows = await db.query(`SELECT * FROM t_checkin ${where} ORDER BY checkin_date DESC`, params);
    return res.json(ok(rows));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
```

```
});

/**
 * 打卡详情
 * GET /api/checkin/detail/:id
 */
router.get('/detail/:id', authRequired, async (req, res) =&gt; {
  try {
    const rows = await db.query('SELECT * FROM t_checkin WHERE id = ?', [req.params.id]);
    if (rows.length === 0) return res.json(fail('打卡记录不存在', 404));
    return res.json(ok(rows[0]));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 审核打卡（家长/管理员）
 * POST /api/checkin/review/:id
 */
router.post('/review/:id', authRequired, async (req, res) =&gt; {
  try {
    const { result, remark } = req.body || {};
    if (!['approved', 'rejected'].includes(result)) {
      return res.json(fail('审核结果非法'));
    }
    await db.query(
      'UPDATE t_checkin SET audit_status = ?, audit_remark = ?, auditor_id = ?, audited_at = NOW() WHERE
id = ?',
      [result, remark || '', req.user.id, req.params.id]
    );
    return res.json(ok(null, '审核完成'));
  } catch (e) {
    return res.json(fail('审核失败', 500));
  }
});

/**
 * 删除打卡
 * POST /api/checkin/delete/:id
 */
router.post('/delete/:id', authRequired, async (req, res) =&gt; {
  try {
    await db.query('DELETE FROM t_checkin WHERE id = ?', [req.params.id]);
    return res.json(ok(null, '删除成功'));
  } catch (e) {
    return res.json(fail('删除失败', 500));
  }
});

module.exports = router;
```

```
// ============ 文件：server/routes/index.js ============
/**
 * 路由聚合模块
 * 将各业务路由挂载到统一前缀 /api。
 */
const express = require('express');
const authRouter = require('./auth');
const userRouter = require('./user');
const taskRouter = require('./task');
const checkinRouter = require('./checkin');
const pointRouter = require('./point');
const badgeRouter = require('./badge');
const adminRouter = require('./admin');

const router = express.Router();

router.use('/auth', authRouter);
router.use('/user', userRouter);
router.use('/task', taskRouter);
router.use('/checkin', checkinRouter);
router.use('/point', pointRouter);
router.use('/badge', badgeRouter);
router.use('/admin', adminRouter);

module.exports = router;
// ============ 文件：server/routes/point.js ============
/**
 * 积分路由模块
 * 提供积分账本查询与任务完成加分逻辑。
 */
const express = require('express');
const db = require('../db');
const { ok, fail } = require('../utils/response');
const { authRequired } = require('../middleware/auth');
const pointService = require('../services/pointService');

const router = express.Router();

/**
 * 查询积分明细（账本流水）
 * GET /api/point/logs
 */
router.get('/logs', authRequired, async (req, res) =&gt; {
  try {
    const rows = await db.query(
      'SELECT id, student_id, delta, reason, remark, created_at FROM t_point_log WHERE student_id = ?
ORDER BY created_at DESC LIMIT 50',
      [req.user.id]
    );
    return res.json(ok(rows));
  } catch (e) {
```

```
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 查询当前积分余额
 * GET /api/point/balance
 */
router.get('/balance', authRequired, async (req, res) =&gt; {
  try {
    const rows = await db.query('SELECT xp FROM t_student_profile WHERE student_id = ?', [req.user.id]);
    const xp = rows.length &gt; 0 ? rows[0].xp : 0;
    return res.json(ok({ xp, level: pointService.levelOf(xp) }));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 管理员调整积分
 * POST /api/point/adjust
 */
router.post('/adjust', authRequired, async (req, res) =&gt; {
  try {
    const { studentId, delta, reason } = req.body || {};
    if (!studentId || !delta || !reason) return res.json(fail('参数不完整'));
    await pointService.adjust(studentId, delta, reason, req.user.id);
    return res.json(ok(null, '调整成功'));
  } catch (e) {
    return res.json(fail('调整失败', 500));
  }
});

module.exports = router;
// ============ 文件：server/routes/task.js ============
/**
 * 任务路由模块
 * 提供学习任务的增删改查、状态流转与派发管理。
 */
const express = require('express');
const db = require('../db');
const { ok, fail } = require('../utils/response');
const { authRequired } = require('../middleware/auth');
const taskService = require('../services/taskService');

const router = express.Router();

/**
 * 任务列表（支持分页与状态筛选）
 * GET /api/task/list
```

```
 */
router.get('/list', authRequired, async (req, res) =&gt; {
  try {
    const pageNo = Math.max(parseInt(req.query.pageNo, 10) || 1, 1);
    const pageSize = Math.min(Math.max(parseInt(req.query.pageSize, 10) || 20, 1), 100);
    const status = req.query.status;
    const subject = req.query.subject;
    const keyword = req.query.keyword;

    const conds = [];
    const params = [];
    if (status) {
      conds.push('t.status = ?');
      params.push(status);
    }
    if (subject) {
      conds.push('t.subject = ?');
      params.push(subject);
    }
    if (keyword) {
      conds.push('(t.title LIKE ? OR t.description LIKE ?)');
      params.push(`%${keyword}%`, `%${keyword}%`);
    }
    if (req.user.role !== 'admin') {
      conds.push('(t.creator_id = ? OR EXISTS (SELECT 1 FROM t_task_assignee a WHERE a.task_id = t.id AND
a.student_id = ?))');
      params.push(req.user.id, req.user.id);
    }

    const where = conds.length &gt; 0 ? `WHERE ${conds.join(' AND ')}` : '';
    const offset = (pageNo - 1) * pageSize;

    const countRows = await db.query(`SELECT COUNT(*) AS total FROM t_task t ${where}`, params);
    const total = countRows[0] ? countRows[0].total : 0;
    const list = await db.query(
      `SELECT t.*, u.name AS creator_name
         FROM t_task t
         LEFT JOIN t_user u ON u.id = t.creator_id
         ${where}
         ORDER BY t.created_at DESC
         LIMIT ? OFFSET ?`,
      [...params, pageSize, offset]
    );

    return res.json(ok({ list, total, pageNo, pageSize }));
  } catch (e) {
    return res.json(fail('查询任务失败', 500));
  }
});

/**
```

```
 * 任务详情
 * GET /api/task/detail/:id
 */
router.get('/detail/:id', authRequired, async (req, res) =&gt; {
  try {
    const task = await taskService.getTaskDetail(req.params.id);
    if (!task) return res.json(fail('任务不存在', 404));
    return res.json(ok(task));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 创建任务
 * POST /api/task/create
 */
router.post('/create', authRequired, async (req, res) =&gt; {
  try {
    const body = req.body || {};
    if (!body.title) return res.json(fail('任务标题不能为空'));

    const taskId = await taskService.createTask(body, req.user);
    return res.json(ok({ taskId }, '创建成功'));
  } catch (e) {
    return res.json(fail('创建失败', 500));
  }
});

/**
 * 更新任务
 * POST /api/task/update/:id
 */
router.post('/update/:id', authRequired, async (req, res) =&gt; {
  try {
    const ok1 = await taskService.updateTask(req.params.id, req.body || {}, req.user);
    if (!ok1) return res.json(fail('任务不存在或无权限', 404));
    return res.json(ok(null, '保存成功'));
  } catch (e) {
    return res.json(fail('更新失败', 500));
  }
});

/**
 * 删除任务
 * POST /api/task/delete/:id
 */
router.post('/delete/:id', authRequired, async (req, res) =&gt; {
  try {
    const ok1 = await taskService.deleteTask(req.params.id, req.user);
```

```
    if (!ok1) return res.json(fail('任务不存在或无权限', 404));
    return res.json(ok(null, '删除成功'));
  } catch (e) {
    return res.json(fail('删除失败', 500));
  }
});

/**
 * 更新任务状态（todo / doing / done）
 * POST /api/task/status/:id
 */
router.post('/status/:id', authRequired, async (req, res) =&gt; {
  try {
    const { status } = req.body || {};
    if (!['todo', 'doing', 'done'].includes(status)) {
      return res.json(fail('状态值非法'));
    }
    await db.query('UPDATE t_task SET status = ?, updated_at = NOW() WHERE id = ?', [status,
req.params.id]);
    return res.json(ok(null, '状态更新成功'));
  } catch (e) {
    return res.json(fail('更新失败', 500));
  }
});

module.exports = router;
// ============ 文件：server/routes/user.js ============
/**
 * 用户路由模块
 * 提供用户资料查询、修改与密码修改能力。
 */
const express = require('express');
const bcrypt = require('bcryptjs');
const db = require('../db');
const { ok, fail } = require('../utils/response');
const { requiredString } = require('../utils/validate');
const { authRequired } = require('../middleware/auth');

const router = express.Router();

/**
 * 获取个人资料
 * GET /api/user/profile
 */
router.get('/profile', authRequired, async (req, res) =&gt; {
  try {
    const rows = await db.query(
      'SELECT id, account, name, role, avatar, gender, school, grade, created_at FROM t_user WHERE id = ?',
      [req.user.id]
    );
    if (rows.length === 0) return res.json(fail('用户不存在', 404));
```

```
    return res.json(ok(rows[0]));
  } catch (e) {
    return res.json(fail('查询失败', 500));
  }
});

/**
 * 更新个人资料
 * POST /api/user/profile
 */
router.post('/profile', authRequired, async (req, res) =&gt; {
  try {
    const { name, avatar, gender, school, grade } = req.body || {};
    const fields = [];
    const params = [];
    const allowMap = { name, avatar, gender, school, grade };
    for (const [key, val] of Object.entries(allowMap)) {
      if (val !== undefined) {
        fields.push(`${key} = ?`);
        params.push(val);
      }
    }
    if (fields.length === 0) return res.json(fail('没有需要更新的字段'));

    params.push(req.user.id);
    await db.query(`UPDATE t_user SET ${fields.join(', ')} WHERE id = ?`, params);
    return res.json(ok(null, '保存成功'));
  } catch (e) {
    return res.json(fail('更新失败', 500));
  }
});

/**
 * 修改密码
 * POST /api/user/password
 */
router.post('/password', authRequired, async (req, res) =&gt; {
  try {
    const { oldPassword, newPassword } = req.body || {};
    if (!oldPassword || !newPassword) return res.json(fail('参数不完整'));

    const rows = await db.query('SELECT password FROM t_user WHERE id = ?', [req.user.id]);
    if (rows.length === 0) return res.json(fail('用户不存在', 404));

    const matched = await bcrypt.compare(oldPassword, rows[0].password);
    if (!matched) return res.json(fail('原密码错误'));

    const hash = await bcrypt.hash(newPassword, 10);
    await db.query('UPDATE t_user SET password = ? WHERE id = ?', [hash, req.user.id]);
    return res.json(ok(null, '密码修改成功'));
```

```
  } catch (e) {
    return res.json(fail('修改失败', 500));
  }
});

module.exports = router;
// ============ 文件：server/config.js ============
/**
 * 全局配置模块
 * 集中管理服务端口、数据库连接、密钥等运行参数。
 * 所有敏感配置均通过环境变量注入，避免硬编码泄露。
 */
const path = require('path');
const dotenv = require('dotenv');

dotenv.config({ path: path.join(__dirname, '.env') });

const env = process.env.NODE_ENV || 'development';

const config = {
  env: env,
  isProd: env === 'production',
  port: parseInt(process.env.PORT || '3000', 10),
  host: process.env.HOST || '0.0.0.0',

  // 数据库连接配置
  db: {
    host: process.env.DB_HOST || '127.0.0.1',
    port: parseInt(process.env.DB_PORT || '3306', 10),
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '',
    database: process.env.DB_NAME || 'study_checkin',
    connectionLimit: parseInt(process.env.DB_CONNECTION_LIMIT || '10', 10),
    timezone: '+08:00',
    charset: 'utf8mb4'
  },

  // JWT 签名配置
  jwt: {
    secret: process.env.JWT_SECRET || 'change-me-in-production',
    expiresIn: process.env.JWT_EXPIRES || '12h'
  },

  // 文件上传目录
  uploadDir: path.join(__dirname, 'uploads'),

  // 接口限流配置
  rateLimit: {
    windowMs: 15 * 60 * 1000,
    max: 1000
```

```
  }
};

module.exports = config;
// ============ 文件：server/db.js ============
/**
 * 数据库连接池模块
 * 使用 mysql2 连接池管理 MySQL 连接，提供统一的查询封装。
 */
const mysql = require('mysql2/promise');
const config = require('./config');

const pool = mysql.createPool({
  host: config.db.host,
  port: config.db.port,
  user: config.db.user,
  password: config.db.password,
  database: config.db.database,
  connectionLimit: config.db.connectionLimit,
  timezone: config.db.timezone,
  charset: config.db.charset,
  waitForConnections: true,
  queueLimit: 0
});

/**
 * 执行 SQL 查询
 * @param {string} sql SQL 语句
 * @param {Array} params 参数列表
 * @returns {Promise&lt;Array&gt;} 查询结果行
 */
async function query(sql, params) {
  const [rows] = await pool.execute(sql, params || []);
  return rows;
}

/**
 * 事务封装
 * @param {Function} fn 事务内执行的异步函数，接收 conn 参数
 * @returns {Promise&lt;*&gt;} 事务结果
 */
async function transaction(fn) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const result = await fn(conn);
    await conn.commit();
    return result;
  } catch (err) {
    await conn.rollback();
```

```
    throw err;
  } finally {
    conn.release();
  }
}

/**
 * 校验数据库连接
 */
async function ping() {
  await pool.query('SELECT 1');
}

module.exports = { pool, query, transaction, ping };
// ============ 文件：server/middleware/auth.js ============
/**
 * 鉴权中间件
 * 校验请求头中的访问令牌，并将用户信息挂载到 req.user。
 */
const jwt = require('../utils/jwt');
const { fail } = require('../utils/response');
const db = require('../db');

async function authRequired(req, res, next) {
  try {
    const token = jwt.parseBearer(req.headers.authorization || '');
    if (!token) {
      return res.json(fail('未登录或令牌缺失', 401));
    }
    const payload = jwt.verify(token);
    const rows = await db.query('SELECT id, name, role, status FROM t_user WHERE id = ?', [payload.uid]);
    if (rows.length === 0) {
      return res.json(fail('用户不存在', 401));
    }
    const user = rows[0];
    if (user.status !== 1) {
      return res.json(fail('账号已被禁用', 403));
    }
    req.user = user;
    next();
  } catch (err) {
    if (err.name === 'JsonWebTokenError' || err.name === 'TokenExpiredError') {
      return res.json(fail('登录状态失效，请重新登录', 401));
    }
    return res.json(fail('鉴权异常', 500));
  }
}

function adminOnly(req, res, next) {
  if (!req.user || req.user.role !== 'admin') {
```

```
    return res.json(fail('无管理员权限', 403));
  }
  next();
}

module.exports = { authRequired, adminOnly };
// ============ 文件：server/middleware/errorHandler.js ============
/**
 * 错误处理中间件
 * 统一捕获异常并返回规范的错误响应。
 */
const logger = require('../utils/logger');
const { fail } = require('../utils/response');

function notFound(req, res) {
  res.status(404).json(fail('接口不存在', 404));
}

function errorHandler(err, req, res, next) {
  logger.error('unhandled error', err);
  if (res.headersSent) {
    return next(err);
  }
  const message = process.env.NODE_ENV === 'production' ? '服务器内部错误' : err.message;
  res.status(500).json(fail(message, 500));
}

module.exports = { notFound, errorHandler };
// ============ 文件：server/middleware/rateLimit.js ============
/**
 * 简单限流中间件
 * 基于进程内滑动窗口实现，防止接口被暴力调用。
 */
const config = require('../config');
const { fail } = require('../utils/response');

const buckets = new Map();

function rateLimit(keyFn, options) {
  const windowMs = (options &amp;&amp; options.windowMs) || config.rateLimit.windowMs;
  const max = (options &amp;&amp; options.max) || config.rateLimit.max;

  return function (req, res, next) {
    const key = keyFn(req);
    const now = Date.now();
    if (!buckets.has(key)) {
      buckets.set(key, []);
    }
    const arr = buckets.get(key);
    while (arr.length &gt; 0 &amp;&amp; now - arr[0] &gt; windowMs) {
```

```
      arr.shift();
    }
    if (arr.length &gt;= max) {
      return res.json(fail('请求过于频繁，请稍后再试', 429));
    }
    arr.push(now);
    next();
  };
}

module.exports = { rateLimit };
// ============ 文件：server/package.json ============
{
  "name": "study-checkin-server",
  "version": "1.2.0",
  "description": "学习打卡管理平台后端服务",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "express": "^4.19.0",
    "mysql2": "^3.11.0",
    "jsonwebtoken": "^9.0.2",
    "bcryptjs": "^2.4.3",
    "multer": "^1.4.5-lts.1",
    "uuid": "^9.0.1"
  },
  "engines": {
    "node": "&gt;=18"
  }
}
// ============ 文件：server/services/auditService.js ============
/**
 * 审计服务模块
 * 记录关键操作日志，便于追溯与合规审查。
 */
const db = require('../db');

async function log(operatorId, action, detail, ip) {
  try {
    await db.query(
      'INSERT INTO t_audit_log (operator_id, action, detail, ip, created_at) VALUES (?, ?, ?, ?, NOW())',
      [operatorId, action, detail ? JSON.stringify(detail) : null, ip || '']
    );
  } catch (e) {
    // 审计失败不阻断主流程
  }
}
```

```
async function list(pageNo = 1, pageSize = 20) {
  const offset = (pageNo - 1) * pageSize;
  const rows = await db.query(
    `SELECT l.*, u.name AS operator_name
       FROM t_audit_log l
       LEFT JOIN t_user u ON u.id = l.operator_id
      ORDER BY l.created_at DESC
      LIMIT ? OFFSET ?`,
    [pageSize, offset]
  );
  const totalRows = await db.query('SELECT COUNT(*) AS c FROM t_audit_log');
  return { list: rows, total: totalRows[0].c };
}

module.exports = { log, list };
// ============ 文件：server/services/checkinService.js ============
/**
 * 打卡服务模块
 * 封装打卡提交与自动加分逻辑。
 */
const db = require('../db');
const { transaction } = require('../db');
const pointService = require('./pointService');

async function createCheckin(body, user) {
  const { taskId, checkinDate, note, images, voiceUrl } = body;
  return transaction(async (conn) =&gt; {
    const result = await conn.execute(
      `INSERT INTO t_checkin (task_id, student_id, checkin_date, note, images, voice_url, audit_status,
created_at)
       VALUES (?, ?, ?, ?, ?, ?, 'pending', NOW())`,
      [taskId, user.id, checkinDate, note || '', JSON.stringify(images || []), voiceUrl || null]
    );
    const checkinId = result[0].insertId;

    // 任务首次打卡后自动流转为进行中
    await conn.execute(
      "UPDATE t_task SET status = 'doing', updated_at = NOW() WHERE id = ? AND status = 'todo'",
      [taskId]
    );

    // 家长/管理员提交的打卡自动审核通过
    if (['parent', 'admin'].includes(user.role)) {
      await conn.execute(
        "UPDATE t_checkin SET audit_status = 'approved', auditor_id = ?, audited_at = NOW() WHERE id = ?",
        [user.id, checkinId]
      );
      await pointService.grantForCheckin(conn, user.id, checkinId);
    }
    return checkinId;
```

```
  });
}

async function approveCheckin(conn, checkinId, auditorId) {
  await conn.execute(
    "UPDATE t_checkin SET audit_status = 'approved', auditor_id = ?, audited_at = NOW() WHERE id = ?",
    [auditorId, checkinId]
  );
}

async function getByTaskAndDate(taskId, studentId, date) {
  const rows = await db.query(
    'SELECT * FROM t_checkin WHERE task_id = ? AND student_id = ? AND checkin_date = ?',
    [taskId, studentId, date]
  );
  return rows[0] || null;
}

module.exports = { createCheckin, approveCheckin, getByTaskAndDate };
// ============ 文件：server/services/pointService.js ============
/**
 * 积分服务模块
 * 积分账本式管理：每次变动写一条流水，余额从账本累加。
 */
const db = require('../db');

const LEVELS = [
  { level: 1, xp: 0 },
  { level: 2, xp: 50 },
  { level: 3, xp: 150 },
  { level: 4, xp: 300 },
  { level: 5, xp: 500 },
  { level: 6, xp: 800 },
  { level: 7, xp: 1200 },
  { level: 8, xp: 1800 },
  { level: 9, xp: 2500 },
  { level: 10, xp: 3200 }
];

function levelOf(xp) {
  let lv = LEVELS[0];
  for (const item of LEVELS) {
    if (xp &gt;= item.xp) lv = item;
    else break;
  }
  return lv;
}

async function addLog(conn, studentId, delta, reason, remark) {
  await conn.execute(
```

```
    'INSERT INTO t_point_log (student_id, delta, reason, remark, created_at) VALUES (?, ?, ?, ?, NOW())',
    [studentId, delta, reason, remark || '']
  );
}

async function adjust(studentId, delta, reason, operatorId) {
  const conn = await db.pool.getConnection();
  try {
    await conn.beginTransaction();
    await addLog(conn, studentId, delta, reason, `operator:${operatorId}`);
    await syncBalance(conn, studentId);
    await conn.commit();
  } catch (e) {
    await conn.rollback();
    throw e;
  } finally {
    conn.release();
  }
}

async function syncBalance(conn, studentId) {
  const rows = await conn.execute(
    'SELECT COALESCE(SUM(delta), 0) AS total FROM t_point_log WHERE student_id = ?',
    [studentId]
  );
  const total = rows[0][0].total;
  await conn.execute(
    `INSERT INTO t_student_profile (student_id, xp) VALUES (?, ?)
     ON DUPLICATE KEY UPDATE xp = VALUES(xp)`,
    [studentId, total]
  );
}

async function grantForCheckin(conn, studentId, checkinId) {
  await addLog(conn, studentId, 10, 'checkin_approved', `checkin:${checkinId}`);
  await syncBalance(conn, studentId);
}

module.exports = { levelOf, adjust, addLog, grantForCheckin, syncBalance };
// ============ 文件：server/services/statsService.js ============
/**
 * 统计服务模块
 * 汇总学习仪表盘所需的各项统计指标。
 */
const db = require('../db');

async function studentStats(studentId) {
  const [taskTotal, doneTotal, checkinTotal, approvedTotal] = await Promise.all([
    db.query('SELECT COUNT(*) AS c FROM t_task_assignee WHERE student_id = ?', [studentId]),
    db.query(
```

```
      "SELECT COUNT(*) AS c FROM t_task_assignee a JOIN t_task t ON t.id = a.task_id WHERE a.student_id = ?
AND t.status = 'done'",
      [studentId]
    ),
    db.query('SELECT COUNT(*) AS c FROM t_checkin WHERE student_id = ?', [studentId]),
    db.query(
      "SELECT COUNT(*) AS c FROM t_checkin WHERE student_id = ? AND audit_status = 'approved'",
      [studentId]
    )
  ]);
  return {
    taskTotal: taskTotal[0].c,
    doneTotal: doneTotal[0].c,
    checkinTotal: checkinTotal[0].c,
    approvedTotal: approvedTotal[0].c
  };
}

async function weeklyTrend(studentId, days = 7) {
  const since = new Date(Date.now() - days * 86400000);
  const sinceKey = since.toISOString().slice(0, 10);
  const rows = await db.query(
    `SELECT checkin_date, COUNT(*) AS cnt FROM t_checkin
      WHERE student_id = ? AND checkin_date &gt;= ? AND audit_status = 'approved'
      GROUP BY checkin_date ORDER BY checkin_date`,
    [studentId, sinceKey]
  );
  return rows;
}

async function subjectDistribution(studentId) {
  const rows = await db.query(
    `SELECT t.subject, COUNT(*) AS cnt
       FROM t_task_assignee a
       JOIN t_task t ON t.id = a.task_id
      WHERE a.student_id = ? AND t.subject IS NOT NULL
      GROUP BY t.subject`,
    [studentId]
  );
  return rows;
}

module.exports = { studentStats, weeklyTrend, subjectDistribution };
// ============ 文件：server/services/streakService.js ============
/**
 * 连续打卡服务模块
 * 计算学生连续打卡天数，并在达标时触发徽章解锁。
 */
const db = require('../db');
const { transaction } = require('../db');
```

```
/**
 * 计算连续打卡天数（按打卡记录倒序扫描）
 */
async function calcStreak(studentId) {
  const rows = await db.query(
    `SELECT DISTINCT checkin_date FROM t_checkin
     WHERE student_id = ? AND audit_status = 'approved'
     ORDER BY checkin_date DESC`,
    [studentId]
  );
  const dates = rows.map((r) =&gt; r.checkin_date);
  if (dates.length === 0) return 0;

  let streak = 1;
  const today = new Date();
  const todayKey = dateKey(today);
  let prev = dates[0];
  if (prev !== todayKey) {
    const yesterdayKey = dateKey(new Date(today.getTime() - 86400000));
    if (prev !== yesterdayKey) return 0;
  }
  for (let i = 1; i &lt; dates.length; i++) {
    const diff = dayDiff(dates[i - 1], dates[i]);
    if (diff === 1) {
      streak += 1;
    } else {
      break;
    }
  }
  return streak;
}

function dateKey(d) {
  return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2,
'0')}`;
}

function dayDiff(a, b) {
  const da = new Date(a + 'T00:00:00');
  const db2 = new Date(b + 'T00:00:00');
  return Math.round((db2 - da) / 86400000);
}

/**
 * 更新学生连续打卡天数
 */
async function updateStreak(studentId) {
  const streak = await calcStreak(studentId);
  await db.query(
    'UPDATE t_student_profile SET streak = ? WHERE student_id = ?',
    [streak, studentId]
```

```
  );
  return streak;
}

/**
 * 连续打卡达标解锁徽章
 */
async function unlockStreakBadge(studentId, streak) {
  const keys = {
    7: 'seven_days',
    21: 'three_weeks',
    30: 'month_master'
  };
  const key = keys[streak];
  if (!key) return;
  await transaction(async (conn) =&gt; {
    await conn.execute(
      'INSERT IGNORE INTO t_badge_unlock (student_id, badge_key) VALUES (?, ?)',
      [studentId, key]
    );
  });
}

module.exports = { calcStreak, updateStreak, unlockStreakBadge };
// ============ 文件：server/services/taskService.js ============
/**
 * 任务服务模块
 * 封装任务创建、更新、删除等核心业务逻辑。
 */
const db = require('../db');
const { transaction } = require('../db');

const STATUS = ['todo', 'doing', 'done'];

async function getTaskDetail(id) {
  const rows = await db.query(
    `SELECT t.*, u.name AS creator_name
       FROM t_task t
       LEFT JOIN t_user u ON u.id = t.creator_id
      WHERE t.id = ?`,
    [id]
  );
  if (rows.length === 0) return null;
  const task = rows[0];
  const assignees = await db.query(
    `SELECT u.id, u.name FROM t_task_assignee a JOIN t_user u ON u.id = a.student_id WHERE a.task_id = ?`,
    [id]
  );
  task.assignees = assignees;
  return task;
```

```
}

async function createTask(body, user) {
  const {
    title, subject, description, startDate, deadline,
    score, images, assigneeIds = []
  } = body;

  return transaction(async (conn) =&gt; {
    const result = await conn.execute(
      `INSERT INTO t_task (title, subject, description, start_date, deadline, score, images, status,
creator_id, created_at, updated_at)
       VALUES (?, ?, ?, ?, ?, ?, ?, 'todo', ?, NOW(), NOW())`,
      [title, subject, description || '', startDate, deadline, score || 0, JSON.stringify(images || []),
user.id]
    );
    const taskId = result[0].insertId;

    const students = user.role === 'admin' &amp;&amp; assigneeIds.length &gt; 0 ? assigneeIds : [user.id];
    for (const sid of students) {
      await conn.execute('INSERT INTO t_task_assignee (task_id, student_id) VALUES (?, ?)', [taskId, sid]);
    }
    return taskId;
  });
}

async function updateTask(id, body, user) {
  const fields = [];
  const params = [];
  const allowMap = {
    title: body.title,
    subject: body.subject,
    description: body.description,
    startDate: body.startDate,
    deadline: body.deadline,
    score: body.score,
    status: body.status,
    images: body.images === undefined ? undefined : JSON.stringify(body.images)
  };
  for (const [key, val] of Object.entries(allowMap)) {
    if (val !== undefined &amp;&amp; val !== null) {
      fields.push(`${key} = ?`);
      params.push(val);
    }
  }
  if (fields.length === 0) return true;

  const rows = await db.query('SELECT creator_id FROM t_task WHERE id = ?', [id]);
  if (rows.length === 0) return false;
  if (user.role !== 'admin' &amp;&amp; rows[0].creator_id !== user.id) return false;

  params.push(id);
```

```
  await db.query(`UPDATE t_task SET ${fields.join(', ')}, updated_at = NOW() WHERE id = ?`, params);
  return true;
}

async function deleteTask(id, user) {
  const rows = await db.query('SELECT creator_id FROM t_task WHERE id = ?', [id]);
  if (rows.length === 0) return false;
  if (user.role !== 'admin' &amp;&amp; rows[0].creator_id !== user.id) return false;

  await transaction(async (conn) =&gt; {
    await conn.execute('DELETE FROM t_task_assignee WHERE task_id = ?', [id]);
    await conn.execute('DELETE FROM t_checkin WHERE task_id = ?', [id]);
    await conn.execute('DELETE FROM t_task WHERE id = ?', [id]);
  });
  return true;
}

module.exports = { getTaskDetail, createTask, updateTask, deleteTask, STATUS };
// ============ 文件：server/utils/jwt.js ============
/**
 * JWT 工具模块
 * 负责访问令牌的签发与校验，支持用户端与管理端两套令牌。
 */
const jwt = require('jsonwebtoken');
const config = require('../config');

function sign(payload, options) {
  return jwt.sign(payload, config.jwt.secret, {
    expiresIn: config.jwt.expiresIn,
    ...options
  });
}

function verify(token) {
  return jwt.verify(token, config.jwt.secret);
}

function parseBearer(header) {
  if (!header || !header.startsWith('Bearer ')) return null;
  return header.slice(7).trim();
}

module.exports = { sign, verify, parseBearer };
// ============ 文件：server/utils/logger.js ============
/**
 * 日志模块
 * 按级别输出日志，生产环境可接入外部日志采集。
 */
const util = require('util');
```

```
function format(args) {
  return args.map((a) =&gt; {
    if (typeof a === 'string') return a;
    return util.inspect(a, { depth: 5, breakLength: 120 });
  }).join(' ');
}

function ts() {
  return new Date().toISOString();
}

const logger = {
  info(...args) {
    console.log(`[INFO] ${ts()} ${format(args)}`);
  },
  warn(...args) {
    console.warn(`[WARN] ${ts()} ${format(args)}`);
  },
  error(...args) {
    console.error(`[ERROR] ${ts()} ${format(args)}`);
  },
  debug(...args) {
    if (process.env.DEBUG) {
      console.log(`[DEBUG] ${ts()} ${format(args)}`);
    }
  }
};

module.exports = logger;
// ============ 文件：server/utils/pagination.js ============
/**
 * 分页参数工具模块
 * 统一解析与校验分页参数，防止非法输入。
 */
function parsePage(query) {
  const pageNo = Math.max(parseInt(query.pageNo, 10) || 1, 1);
  const pageSize = Math.min(Math.max(parseInt(query.pageSize, 10) || 20, 1), 100);
  return { pageNo, pageSize, offset: (pageNo - 1) * pageSize };
}

function buildPageResult(list, total, pageNo, pageSize) {
  return {
    list,
    total,
    pageNo,
    pageSize,
    hasMore: pageNo * pageSize &lt; total
  };
}
```

```
module.exports = { parsePage, buildPageResult };
// ============ 文件：server/utils/response.js ============
/**
 * 统一响应封装模块
 * 约定前端统一的响应结构：{ code, message, data }
 * code = 0 表示成功，非 0 表示业务错误。
 */
const SUCCESS_CODE = 0;

/**
 * 成功响应
 */
function ok(data, message = 'success') {
  return { code: SUCCESS_CODE, message, data };
}

/**
 * 失败响应
 */
function fail(message, code = 400) {
  return { code, message, data: null };
}

/**
 * 分页数据封装
 */
function page(rows, total, pageNo, pageSize) {
  return {
    list: rows,
    total,
    pageNo,
    pageSize,
    hasMore: pageNo * pageSize &lt; total
  };
}

module.exports = { ok, fail, page, SUCCESS_CODE };
// ============ 文件：server/utils/validate.js ============
/**
 * 参数校验工具模块
 * 提供常用的输入校验函数，防止非法参数进入业务逻辑。
 */
const { fail } = require('./response');

/**
 * 非空字符串校验
 */
function requiredString(value, name) {
  if (typeof value !== 'string' || value.trim().length === 0) {
    return fail(`${name}不能为空`);
```

```
  }
  return null;
}

/**
 * 正整数校验
 */
function positiveInt(value, name) {
  const num = Number(value);
  if (!Number.isInteger(num) || num &lt;= 0) {
    return fail(`${name}必须为正整数`);
  }
  return null;
}

/**
 * 枚举值校验
 */
function inEnum(value, enums, name) {
  if (!enums.includes(value)) {
    return fail(`${name}取值非法`);
  }
  return null;
}

/**
 * 数组校验
 */
function isArray(value, name) {
  if (!Array.isArray(value)) {
    return fail(`${name}必须为数组`);
  }
  return null;
}

module.exports = { requiredString, positiveInt, inEnum, isArray };
// ============ 文件：server/sql/schema.sql ============
-- ============================================
-- 学习打卡管理平台 数据库结构定义
-- MySQL 8.0 / utf8mb4
-- ============================================
SET NAMES utf8mb4;

-- 用户表
CREATE TABLE IF NOT EXISTS t_user (
  id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  account      VARCHAR(64)     NOT NULL,
  password     VARCHAR(128)    NOT NULL,
  name         VARCHAR(32)     NOT NULL,
  role         VARCHAR(16)     NOT NULL DEFAULT 'student',
```

```
  margin-top: 12rpx;
}

.stat-grid {
  display: flex;
  justify-content: space-between;
  margin-bottom: 32rpx;
}

.stat-card {
  width: 23%;
  background: #fff;
  border-radius: 20rpx;
  padding: 28rpx 0;
  text-align: center;
}

.stat-num {
  font-size: 44rpx;
  font-weight: 700;
  color: #4A90D9;
}

.stat-label {
  font-size: 22rpx;
  color: #999;
  margin-top: 8rpx;
}

.card {
  background: #fff;
  border-radius: 24rpx;
  padding: 32rpx;
  margin-bottom: 32rpx;
}

.card-title {
  font-size: 32rpx;
  font-weight: 600;
  margin-bottom: 24rpx;
}

.today-line {
  font-size: 28rpx;
  color: #555;
}

.actions {
  display: flex;
  flex-direction: column;
```

```
}

.btn-primary {
  background: #4A90D9;
  color: #fff;
  border-radius: 44rpx;
  margin-bottom: 24rpx;
}

.btn-plain {
  background: #fff;
  color: #4A90D9;
  border: 2rpx solid #4A90D9;
  border-radius: 44rpx;
}
// ============ 文件：miniprogram/pages/login/login.js ============
/**
 * 登录页面
 * 支持账号密码登录与注册切换。
 */
const api = require('../../utils/api');

Page({
  data: {
    mode: 'login',
    account: '',
    password: '',
    name: '',
    loading: false
  },

  onAccount(e) {
    this.setData({ account: e.detail.value });
  },

  onPassword(e) {
    this.setData({ password: e.detail.value });
  },

  onName(e) {
    this.setData({ name: e.detail.value });
  },

  switchMode() {
    this.setData({ mode: this.data.mode === 'login' ? 'register' : 'login' });
  },

  async onSubmit() {
    const { mode, account, password, name } = this.data;
    if (!account || !password) {
```

```
      wx.showToast({ title: '请填写账号和密码', icon: 'none' });
      return;
    }
    if (mode === 'register' &amp;&amp; !name) {
      wx.showToast({ title: '请填写姓名', icon: 'none' });
      return;
    }
    this.setData({ loading: true });
    try {
      const fn = mode === 'login' ? api.login : api.register;
      const data = await fn({ account, password, name });
      getApp().setSession(data.token, data);
      wx.switchTab({ url: '/pages/index/index' });
    } catch (e) {
      // 已在封装层提示
    } finally {
      this.setData({ loading: false });
    }
  }
});
// ============ 文件：miniprogram/pages/login/login.wxml ============
&lt;view class="page"&gt;
  &lt;view class="login-title"&gt;学习打卡&lt;/view&gt;
  &lt;view class="form"&gt;
    &lt;input class="input" placeholder="账号" value="{{account}}" bindinput="onAccount" /&gt;
    &lt;input
      class="input"
      placeholder="密码"
      password
      value="{{password}}"
      bindinput="onPassword"
    /&gt;
    &lt;input
      wx:if="{{mode === 'register'}}"
      class="input"
      placeholder="姓名"
      value="{{name}}"
      bindinput="onName"
    /&gt;
    &lt;button class="btn-primary" loading="{{loading}}" bindtap="onSubmit"&gt;
      {{mode === 'login' ? '登 录' : '注 册'}}
    &lt;/button&gt;
    &lt;view class="switch" bindtap="switchMode"&gt;
      {{mode === 'login' ? '还没有账号？去注册' : '已有账号？去登录'}}
    &lt;/view&gt;
  &lt;/view&gt;
&lt;/view&gt;
// ============ 文件：miniprogram/pages/mine/mine.js ============
/**
 * 我的页面
```

```
 * 展示个人信息、积分等级、徽章墙，并提供登出入口。
 */
const api = require('../../utils/api');

Page({
  data: {
    profile: null,
    level: 0,
    xp: 0,
    nextXp: 0,
    badges: []
  },

  onShow() {
    this.refresh();
  },

  async refresh() {
    try {
      const profile = await api.getProfile();
      const balance = await api.pointBalance();
      const badges = await api.myBadges();
      const next = (balance.level &amp;amp;&amp;amp; balance.level.xp) || 0;
      this.setData({
        profile,
        level: balance.level ? balance.level.level : 1,
        xp: balance.xp || 0,
        nextXp: next,
        badges
      });
    } catch (e) {
      // 登录态失效时跳转登录
    }
  },

  onLogin() {
    wx.navigateTo({ url: '/pages/login/login' });
  },

  onLogout() {
    wx.showModal({
      title: '提示',
      content: '确认退出登录吗？',
      success: (res) =&gt; {
        if (res.confirm) {
          const app = getApp();
          app.clearSession();
          this.refresh();
        }
      }
```

```
    });
  },

  goBadges() {
    wx.navigateTo({ url: '/pages/badge/badge' });
  }
});
// ============ 文件：miniprogram/pages/mine/mine.wxml ============
&lt;view class="page"&gt;
  &lt;view class="profile-card" wx:if="{{profile}}"&gt;
    &lt;view class="profile-name"&gt;{{profile.name}}&lt;/view&gt;
    &lt;view class="profile-role"&gt;{{profile.role}}&lt;/view&gt;
    &lt;view class="level-badge"&gt;Lv.{{level}}&lt;/view&gt;
    &lt;view class="xp-line"&gt;
      &lt;text&gt;积分 {{xp}}&lt;/text&gt;
      &lt;text&gt;距下一级还需 {{nextXp - xp}} 分&lt;/text&gt;
    &lt;/view&gt;
  &lt;/view&gt;

  &lt;view class="card"&gt;
    &lt;view class="card-title"&gt;我的徽章&lt;/view&gt;
    &lt;view class="badge-grid"&gt;
      &lt;view class="badge-item" wx:for="{{badges}}" wx:key="key"&gt;
        &lt;view class="badge-icon {{item.unlocked ? '' : 'badge-locked'}}"&gt;{{item.unlocked ? '★' :
'☆'}}&lt;/view&gt;
        &lt;view class="badge-name"&gt;{{item.name}}&lt;/view&gt;
        &lt;view class="badge-desc"&gt;{{item.desc}}&lt;/view&gt;
      &lt;/view&gt;
    &lt;/view&gt;
  &lt;/view&gt;

  &lt;button class="btn-plain" bindtap="onLogout" wx:if="{{profile}}"&gt;退出登录&lt;/button&gt;
  &lt;button class="btn-primary" bindtap="onLogin" wx:else&gt;去登录&lt;/button&gt;
&lt;/view&gt;
// ============ 文件：miniprogram/pages/mine/mine.wxss ============
.page {
  padding: 32rpx;
  background: #F5F6FA;
  min-height: 100vh;
}

.profile-card {
  background: linear-gradient(135deg, #4A90D9, #7F5BE6);
  border-radius: 24rpx;
  padding: 48rpx 40rpx;
  color: #fff;
  margin-bottom: 32rpx;
}

.profile-name {
  font-size: 40rpx;
```

```
  font-weight: 600;
}

.profile-role {
  font-size: 24rpx;
  opacity: 0.85;
  margin-top: 8rpx;
}

.level-badge {
  display: inline-block;
  margin-top: 24rpx;
  background: rgba(255, 255, 255, 0.2);
  padding: 8rpx 24rpx;
  border-radius: 32rpx;
  font-size: 26rpx;
}

.xp-line {
  margin-top: 24rpx;
  display: flex;
  justify-content: space-between;
  font-size: 26rpx;
}

.card {
  background: #fff;
  border-radius: 24rpx;
  padding: 32rpx;
  margin-bottom: 32rpx;
}

.card-title {
  font-size: 32rpx;
  font-weight: 600;
  margin-bottom: 24rpx;
}

.badge-grid {
  display: flex;
  flex-wrap: wrap;
}

.badge-item {
  width: 33.3%;
  text-align: center;
  margin-bottom: 32rpx;
}

.badge-icon {
```

```
  font-size: 56rpx;
  color: #F5A623;
}

.badge-locked {
  color: #ccc;
}

.badge-name {
  font-size: 26rpx;
  margin-top: 8rpx;
}

.badge-desc {
  font-size: 22rpx;
  color: #999;
  margin-top: 4rpx;
}

.btn-primary {
  background: #4A90D9;
  color: #fff;
  border-radius: 44rpx;
}

.btn-plain {
  background: #fff;
  color: #4A90D9;
  border: 2rpx solid #4A90D9;
  border-radius: 44rpx;
}
// ============ 文件：miniprogram/pages/task-edit/task-edit.js ============
/**
 * 任务编辑页
 * 支持创建与编辑学习任务。
 */
const api = require('../../utils/api');

const SUBJECTS = ['语文', '数学', '英语', '阅读', '运动', '音乐', '美术', '其他'];

Page({
  data: {
    id: null,
    form: {
      title: '',
      subject: '',
      description: '',
      score: 0,
      deadline: ''
    },
```

```
  subjects: SUBJECTS,
  saving: false
},

onLoad(options) {
  if (options.id) {
    this.setData({ id: options.id });
    this.load(options.id);
  }
},

async load(id) {
  try {
    const task = await api.taskDetail(id);
    this.setData({
      form: {
        title: task.title || '',
        subject: task.subject || '',
        description: task.description || '',
        score: task.score || 0,
        deadline: task.deadline || ''
      }
    });
  } catch (e) {
    // 已在封装层提示
  }
},

onTitle(e) {
  this.setData({ 'form.title': e.detail.value });
},

onSubject(e) {
  this.setData({ 'form.subject': e.detail.value });
},

onDesc(e) {
  this.setData({ 'form.description': e.detail.value });
},

onScore(e) {
  this.setData({ 'form.score': e.detail.value });
},

onDeadline(e) {
  this.setData({ 'form.deadline': e.detail.value });
},

async onSave() {
  const { id, form } = this.data;
```

```
    if (!form.title.trim()) {
      wx.showToast({ title: '请填写任务标题', icon: 'none' });
      return;
    }
    this.setData({ saving: true });
    try {
      if (id) {
        await api.updateTask(id, form);
      } else {
        await api.createTask(form);
      }
      wx.showToast({ title: '保存成功', icon: 'success' });
      setTimeout(() =&gt; wx.navigateBack(), 800);
    } catch (e) {
      // 已在封装层提示
    } finally {
      this.setData({ saving: false });
    }
  }
});
// ============ 文件：miniprogram/pages/task-edit/task-edit.wxml ============
&lt;view class="page"&gt;
  &lt;view class="card"&gt;
    &lt;view class="field"&gt;
      &lt;view class="label"&gt;标题&lt;/view&gt;
      &lt;input class="input" placeholder="任务标题" value="{{form.title}}" bindinput="onTitle" /&gt;
    &lt;/view&gt;
    &lt;view class="field"&gt;
      &lt;view class="label"&gt;科目&lt;/view&gt;
      &lt;picker range="{{subjects}}" value="{{form.subject}}" bindchange="onSubject"&gt;
        &lt;view class="picker-value"&gt;{{form.subject || '请选择科目'}}&lt;/view&gt;
      &lt;/picker&gt;
    &lt;/view&gt;
    &lt;view class="field"&gt;
      &lt;view class="label"&gt;描述&lt;/view&gt;
      &lt;textarea class="textarea" placeholder="任务描述" value="{{form.description}}" bindinput="onDesc"
maxlength="1000"&gt;&lt;/textarea&gt;
    &lt;/view&gt;
    &lt;view class="field"&gt;
      &lt;view class="label"&gt;分值&lt;/view&gt;
      &lt;input class="input" type="number" placeholder="完成分值" value="{{form.score}}"
bindinput="onScore" /&gt;
    &lt;/view&gt;
    &lt;view class="field"&gt;
      &lt;view class="label"&gt;截止日期&lt;/view&gt;
      &lt;picker mode="date" value="{{form.deadline}}" bindchange="onDeadline"&gt;
        &lt;view class="picker-value"&gt;{{form.deadline || '请选择日期'}}&lt;/view&gt;
      &lt;/picker&gt;
    &lt;/view&gt;
  &lt;/view&gt;
  &lt;button class="btn-primary" loading="{{saving}}" bindtap="onSave"&gt;保存&lt;/button&gt;
&lt;/view&gt;
```

```
// ============ 文件：miniprogram/pages/task-edit/task-edit.wxss ============
.page {
  padding: 32rpx;
  background: #F5F6FA;
  min-height: 100vh;
}

.card {
  background: #fff;
  border-radius: 24rpx;
  padding: 32rpx;
  margin-bottom: 32rpx;
}

.field {
  margin-bottom: 32rpx;
}

.label {
  font-size: 28rpx;
  color: #333;
  font-weight: 600;
  margin-bottom: 12rpx;
}

.input,
.textarea,
.picker-value {
  width: 100%;
  background: #F7F8FA;
  border-radius: 12rpx;
  padding: 20rpx;
  box-sizing: border-box;
  font-size: 28rpx;
}

.textarea {
  height: 200rpx;
}

.btn-primary {
  background: #4A90D9;
  color: #fff;
  border-radius: 44rpx;
}
// ============ 文件：miniprogram/pages/task/task.js ============
/**
 * 任务详情页
 * 展示任务信息、打卡记录，支持提交打卡与状态流转。
 */
```

```
const api = require('../../utils/api');

Page({
  data: {
    id: null,
    task: null,
    checkins: [],
    loading: true,
    checkinText: ''
  },

  onLoad(options) {
    this.setData({ id: options.id });
    this.load();
  },

  async load() {
    this.setData({ loading: true });
    try {
      const task = await api.taskDetail(this.data.id);
      const checkins = await api.listCheckin({ taskId: this.data.id });
      this.setData({ task, checkins, loading: false });
    } catch (e) {
      this.setData({ loading: false });
    }
  },

  // 提交打卡
  async onSubmitCheckin() {
    const content = this.data.checkinText.trim();
    if (!content) {
      wx.showToast({ title: '请填写打卡内容', icon: 'none' });
      return;
    }
    try {
      await api.createCheckin({
        taskId: this.data.id,
        note: content,
        checkinDate: this.today()
      });
      wx.showToast({ title: '打卡成功', icon: 'success' });
      this.setData({ checkinText: '' });
      this.load();
    } catch (e) {
      // 已在封装层提示
    }
  },

  // 完成任务
  async onFinish() {
```

```
    try {
      await api.changeTaskStatus(this.data.id, 'done');
      wx.showToast({ title: '任务已完成', icon: 'success' });
      this.load();
    } catch (e) {
      // 已在封装层提示
    }
  },

  onInput(e) {
    this.setData({ checkinText: e.detail.value });
  },

  today() {
    const now = new Date();
    return `${now.getFullYear()}-${String(now.getMonth() + 1).padStart(2, '0')}-
${String(now.getDate()).padStart(2, '0')}`;
  }
});
// ============ 文件：miniprogram/pages/task/task.wxml ============
&lt;view class="page" wx:if="{{task}}"&gt;
  &lt;view class="card"&gt;
    &lt;view class="task-title"&gt;{{task.title}}&lt;/view&gt;
    &lt;view class="task-meta" wx:if="{{task.subject}}"&gt;科目：{{task.subject}}&lt;/view&gt;
    &lt;view class="task-meta"&gt;状态：{{task.status}}&lt;/view&gt;
    &lt;view class="task-desc" wx:if="{{task.description}}"&gt;{{task.description}}&lt;/view&gt;
    &lt;view class="task-score" wx:if="{{task.score &gt; 0}}"&gt;分值：{{task.score}}&lt;/view&gt;
    &lt;button class="btn-primary" bindtap="onFinish" wx:if="{{task.status !== 'done'}}"&gt;完成任务
&lt;/button&gt;
  &lt;/view&gt;

  &lt;view class="card"&gt;
    &lt;view class="card-title"&gt;打卡记录&lt;/view&gt;
    &lt;view class="checkin-list"&gt;
      &lt;view class="checkin-item" wx:for="{{checkins}}" wx:key="id"&gt;
        &lt;view class="checkin-date"&gt;{{item.checkin_date}}&lt;/view&gt;
        &lt;view class="checkin-note"&gt;{{item.note}}&lt;/view&gt;
        &lt;view class="checkin-status status-{{item.audit_status}}"&gt;{{item.audit_status}}&lt;/view&gt;
      &lt;/view&gt;
      &lt;view wx:if="{{checkins.length === 0}}" class="empty"&gt;暂无打卡记录&lt;/view&gt;
    &lt;/view&gt;
  &lt;/view&gt;

  &lt;view class="card"&gt;
    &lt;view class="card-title"&gt;今日打卡&lt;/view&gt;
    &lt;textarea class="checkin-input" placeholder="写点今天的收获吧…" value="{{checkinText}}"
bindinput="onInput" maxlength="500"&gt;&lt;/textarea&gt;
    &lt;button class="btn-primary" bindtap="onSubmitCheckin"&gt;提交打卡&lt;/button&gt;
  &lt;/view&gt;
&lt;/view&gt;

&lt;view wx:if="{{loading}}" class="loading"&gt;加载中…&lt;/view&gt;
// ============ 文件：miniprogram/pages/task/task.wxss ============
```

```
.page {
  padding: 32rpx;
  background: #F5F6FA;
  min-height: 100vh;
}

.card {
  background: #fff;
  border-radius: 24rpx;
  padding: 32rpx;
  margin-bottom: 32rpx;
}

.task-title {
  font-size: 38rpx;
  font-weight: 600;
}

.task-meta {
  font-size: 26rpx;
  color: #888;
  margin-top: 12rpx;
}

.task-desc {
  font-size: 28rpx;
  color: #555;
  margin-top: 20rpx;
  line-height: 1.6;
}

.task-score {
  font-size: 28rpx;
  color: #F5A623;
  margin-top: 16rpx;
  font-weight: 600;
}

.card-title {
  font-size: 32rpx;
  font-weight: 600;
  margin-bottom: 24rpx;
}

.checkin-item {
  border-bottom: 2rpx solid #f0f0f0;
  padding: 20rpx 0;
}

.checkin-date {
```

```
  font-size: 26rpx;
  color: #999;
}

.checkin-note {
  font-size: 28rpx;
  margin-top: 8rpx;
  line-height: 1.5;
}

.checkin-status {
  display: inline-block;
  font-size: 22rpx;
  margin-top: 12rpx;
  padding: 4rpx 16rpx;
  border-radius: 20rpx;
}

.status-approved {
  color: #52C41A;
  background: #F6FFED;
}

.status-pending {
  color: #FAAD14;
  background: #FFFBE6;
}

.status-rejected {
  color: #FF4D4F;
  background: #FFF1F0;
}

.empty {
  text-align: center;
  color: #bbb;
  padding: 40rpx 0;
}

.checkin-input {
  width: 100%;
  height: 200rpx;
  background: #F7F8FA;
  border-radius: 16rpx;
  padding: 20rpx;
  box-sizing: border-box;
  margin-bottom: 24rpx;
}

.btn-primary {
```

```
  background: #4A90D9;
  color: #fff;
  border-radius: 44rpx;
}

.loading {
  text-align: center;
  color: #999;
  padding: 80rpx 0;
}
// ============ 文件：miniprogram/utils/api.js ============
/**
 * 请求封装模块
 * 统一处理接口请求、令牌注入与错误提示。
 */
const BASE_URL = 'https://api.example.com/api';

function request(method, path, data) {
  const app = getApp();
  return new Promise((resolve, reject) =&gt; {
    wx.request({
      url: BASE_URL + path,
      method,
      data,
      header: {
        'Content-Type': 'application/json',
        Authorization: app.globalData.token ? `Bearer ${app.globalData.token}` : ''
      },
      success(res) {
        const body = res.data;
        if (body &amp;&amp; body.code === 0) {
          resolve(body.data);
        } else if (body &amp;&amp; (body.code === 401 || body.code === 403)) {
          app.clearSession();
          wx.reLaunch({ url: '/pages/mine/mine' });
          reject(body);
        } else {
          wx.showToast({ title: (body &amp;&amp; body.message) || '请求失败', icon: 'none' });
          reject(body);
        }
      },
      fail(err) {
        wx.showToast({ title: '网络异常', icon: 'none' });
        reject(err);
      }
    });
  });
}

const api = {
```

```
  login: (data) =&gt; request('POST', '/auth/login', data),
  register: (data) =&gt; request('POST', '/auth/register', data),
  getProfile: () =&gt; request('GET', '/user/profile'),
  updateProfile: (data) =&gt; request('POST', '/user/profile', data),
  listTask: (params) =&gt; request('GET', `/task/list?${qs(params)}`),
  taskDetail: (id) =&gt; request('GET', `/task/detail/${id}`),
  createTask: (data) =&gt; request('POST', '/task/create', data),
  updateTask: (id, data) =&gt; request('POST', `/task/update/${id}`, data),
  deleteTask: (id) =&gt; request('POST', `/task/delete/${id}`),
  changeTaskStatus: (id, status) =&gt; request('POST', `/task/status/${id}`, { status }),
  createCheckin: (data) =&gt; request('POST', '/checkin/create', data),
  listCheckin: (params) =&gt; request('GET', `/checkin/list?${qs(params)}`),
  reviewCheckin: (id, data) =&gt; request('POST', `/checkin/review/${id}`, data),
  pointLogs: () =&gt; request('GET', '/point/logs'),
  pointBalance: () =&gt; request('GET', '/point/balance'),
  myBadges: () =&gt; request('GET', '/badge/mine')
};

function qs(params) {
  return Object.keys(params || {})
    .filter((k) =&gt; params[k] !== undefined &amp;&amp; params[k] !== '')
    .map((k) =&gt; `${encodeURIComponent(k)}=${encodeURIComponent(params[k])}`)
    .join('&amp;');
}

module.exports = api;
// ============ 文件：miniprogram/utils/util.js ============
/**
 * 通用工具函数模块
 * 提供日期格式化、节流防抖等常用能力。
 */
function pad(n) {
  return n &lt; 10 ? '0' + n : '' + n;
}

function formatDate(d, sep = '-') {
  return [d.getFullYear(), pad(d.getMonth() + 1), pad(d.getDate())].join(sep);
}

function todayKey() {
  return formatDate(new Date());
}

function formatTime(d) {
  return formatDate(d, '-') + ' ' + pad(d.getHours()) + ':' + pad(d.getMinutes());
}

function throttle(fn, wait) {
  let last = 0;
  return function (...args) {
```

```
    const now = Date.now();
    if (now - last &gt;= wait) {
      last = now;
      fn.apply(this, args);
    }
  };
}

function debounce(fn, wait) {
  let timer = null;
  return function (...args) {
    if (timer) clearTimeout(timer);
    timer = setTimeout(() =&gt; fn.apply(this, args), wait);
  };
}

function pick(obj, keys) {
  const out = {};
  keys.forEach((k) =&gt; {
    if (obj[k] !== undefined) out[k] = obj[k];
  });
  return out;
}

module.exports = { pad, formatDate, formatTime, todayKey, throttle, debounce, pick };
// ============ 文件：admin/src/App.jsx ============
import { Routes, Route, Navigate } from 'react-router-dom';
import Login from './pages/Login';
import Dashboard from './pages/Dashboard';
import UserManage from './pages/UserManage';
import TaskManage from './pages/TaskManage';
import StatsPage from './pages/StatsPage';
import RequireAuth from './components/RequireAuth';

export default function App() {
  return (
    &lt;Routes&gt;
      &lt;Route path="/login" element={&lt;Login /&gt;} /&gt;
      &lt;Route
        path="/dashboard"
        element={
          &lt;RequireAuth&gt;
            &lt;Dashboard /&gt;
          &lt;/RequireAuth&gt;
        }
      /&gt;
      &lt;Route
        path="/users"
        element={
          &lt;RequireAuth&gt;
```

```
            &lt;UserManage /&gt;
          &lt;/RequireAuth&gt;
        }
      /&gt;
      &lt;Route
        path="/tasks"
        element={
          &lt;RequireAuth&gt;
            &lt;TaskManage /&gt;
          &lt;/RequireAuth&gt;
        }
      /&gt;
      &lt;Route
        path="/stats"
        element={
          &lt;RequireAuth&gt;
            &lt;StatsPage /&gt;
          &lt;/RequireAuth&gt;
        }
      /&gt;
      &lt;Route path="*" element={&lt;Navigate to="/dashboard" replace /&gt;} /&gt;
    &lt;/Routes&gt;
  );
}
// ============ 文件：admin/src/api.js ============
import axios from 'axios';

const http = axios.create({
  baseURL: '/api',
  timeout: 15000
});

http.interceptors.request.use((config) =&gt; {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

http.interceptors.response.use(
  (res) =&gt; {
    const body = res.data;
    if (body &amp;&amp; body.code === 0) {
      return body.data;
    }
    return Promise.reject(new Error((body &amp;&amp; body.message) || '请求失败'));
  },
  (err) =&gt; {
    if (err.response &amp;&amp; err.response.status === 401) {
```

```
      localStorage.removeItem('token');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export const authApi = {
  login: (data) =&gt; http.post('/auth/login', data)
};

export const userApi = {
  list: (params) =&gt; http.get('/admin/user/list', { params }),
  setStatus: (data) =&gt; http.post('/admin/user/status', data)
};

export const taskApi = {
  list: (params) =&gt; http.get('/task/list', { params }),
  detail: (id) =&gt; http.get(`/task/detail/${id}`),
  create: (data) =&gt; http.post('/task/create', data),
  update: (id, data) =&gt; http.post(`/task/update/${id}`, data),
  remove: (id) =&gt; http.post(`/task/delete/${id}`)
};

export const statsApi = {
  overview: () =&gt; http.get('/admin/stats')
};
// ============ 文件：admin/src/components/RequireAuth.jsx ============
import { Navigate } from 'react-router-dom';

export default function RequireAuth({ children }) {
  const token = localStorage.getItem('token');
  if (!token) {
    return &lt;Navigate to="/login" replace /&gt;;
  }
  return children;
}
// ============ 文件：admin/src/global.css ============
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
    'Helvetica Neue', Arial, 'PingFang SC', 'Microsoft YaHei', sans-serif;
  background: #f0f2f5;
}
```

```
.login-wrap {
  min-height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #4a90d9, #7f5be6);
}

.login-box {
  width: 380px;
  background: #fff;
  border-radius: 12px;
  padding: 40px 32px;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.15);
}

.login-title {
  text-align: center;
  font-size: 22px;
  margin-bottom: 32px;
  color: #333;
}

.logo {
  height: 48px;
  line-height: 48px;
  color: #fff;
  font-size: 16px;
  font-weight: 600;
  text-align: center;
  background: #1f2d4d;
}
// ============ 文件：admin/src/main.jsx ============
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './global.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  &lt;React.StrictMode&gt;
    &lt;BrowserRouter&gt;
      &lt;App /&gt;
    &lt;/BrowserRouter&gt;
  &lt;/React.StrictMode&gt;
);
// ============ 文件：admin/src/pages/CheckinManage.css ============
.checkin-note {
  max-width: 320px;
  white-space: nowrap;
```

```
  overflow: hidden;
  text-overflow: ellipsis;
}
// ============ 文件：admin/src/pages/CheckinManage.jsx ============
import { useEffect, useState } from 'react';
import { Table, Tag, Button, Modal, Input, message } from 'antd';
import { taskApi, checkinApi } from '../api';
import { Space } from 'antd';

export default function CheckinManage() {
  const [list, setList] = useState([]);
  const [loading, setLoading] = useState(false);
  const [pager, setPager] = useState({ pageNo: 1, pageSize: 20, total: 0 });

  const load = async () =&gt; {
    setLoading(true);
    try {
      const data = await checkinApi.list(pager);
      setList(data.list);
      setPager((p) =&gt; ({ ...p, total: data.total }));
    } finally {
      setLoading(false);
    }
  };

  useEffect(() =&gt; {
    load();
  }, [pager.pageNo]);

  const review = async (row, result) =&gt; {
    try {
      await checkinApi.review(row.id, { result });
      message.success('审核完成');
      load();
    } catch (e) {
      message.error(e.message);
    }
  };

  const statusColor = { pending: 'warning', approved: 'success', rejected: 'error' };

  const columns = [
    { title: 'ID', dataIndex: 'id', width: 80 },
    { title: '打卡日期', dataIndex: 'checkin_date' },
    { title: '内容', dataIndex: 'note', ellipsis: true },
    {
      title: '状态',
      dataIndex: 'audit_status',
      render: (s) =&gt; &lt;Tag color={statusColor[s]}&gt;{s}&lt;/Tag&gt;
    },
```

```
    {
      title: '操作',
      render: (_, row) =&gt; (
        &lt;Space&gt;
          &lt;Button
            size="small"
            type="primary"
            disabled={row.audit_status === 'approved'}
            onClick={() =&gt; review(row, 'approved')}
          &gt;
            通过
          &lt;/Button&gt;
          &lt;Button
            size="small"
            danger
            disabled={row.audit_status === 'rejected'}
            onClick={() =&gt; review(row, 'rejected')}
          &gt;
            驳回
          &lt;/Button&gt;
        &lt;/Space&gt;
      )
    }
  ];

  return (
    &lt;div style={{ padding: 16 }}&gt;
      &lt;Table
        rowKey="id"
        loading={loading}
        columns={columns}
        dataSource={list}
        pagination={{
          current: pager.pageNo,
          pageSize: pager.pageSize,
          total: pager.total,
          onChange: (pageNo) =&gt; setPager((p) =&gt; ({ ...p, pageNo }))
        }}
      /&gt;
    &lt;/div&gt;
  );
}
// ============ 文件：admin/src/pages/Dashboard.jsx ============
import { useEffect, useState } from 'react';
import { Card, Row, Col, Statistic, Layout, Menu } from 'antd';
import { useNavigate } from 'react-router-dom';
import { statsApi } from '../api';

const { Header, Content, Sider } = Layout;
```

```
export default function Dashboard() {
  const [stats, setStats] = useState(null);
  const navigate = useNavigate();

  useEffect(() =&gt; {
    statsApi.overview().then(setStats).catch(() =&gt; {});
  }, []);

  const logout = () =&gt; {
    localStorage.removeItem('token');
    navigate('/login');
  };

  return (
    &lt;Layout style={{ minHeight: '100vh' }}&gt;
      &lt;Sider&gt;
        &lt;div className="logo"&gt;学习打卡平台&lt;/div&gt;
        &lt;Menu theme="dark" mode="inline" defaultSelectedKeys={['dashboard']}&gt;
          &lt;Menu.Item key="dashboard" onClick={() =&gt; navigate('/dashboard')}&gt;
            数据概览
          &lt;/Menu.Item&gt;
          &lt;Menu.Item key="users" onClick={() =&gt; navigate('/users')}&gt;
            用户管理
          &lt;/Menu.Item&gt;
          &lt;Menu.Item key="tasks" onClick={() =&gt; navigate('/tasks')}&gt;
            任务管理
          &lt;/Menu.Item&gt;
          &lt;Menu.Item key="stats" onClick={() =&gt; navigate('/stats')}&gt;
            统计报表
          &lt;/Menu.Item&gt;
        &lt;/Menu&gt;
      &lt;/Sider&gt;
      &lt;Layout&gt;
        &lt;Header style={{ background: '#fff', display: 'flex', justifyContent: 'flex-end', alignItems:
'center' }}&gt;
          &lt;Button type="link" onClick={logout}&gt;
            退出登录
          &lt;/Button&gt;
        &lt;/Header&gt;
        &lt;Content style={{ margin: 16 }}&gt;
          &lt;Row gutter={16}&gt;
            &lt;Col span={6}&gt;
              &lt;Card&gt;
                &lt;Statistic title="任务总数" value={stats ? stats.taskTotal : 0} /&gt;
              &lt;/Card&gt;
            &lt;/Col&gt;
            &lt;Col span={6}&gt;
              &lt;Card&gt;
                &lt;Statistic title="打卡总数" value={stats ? stats.checkinTotal : 0} /&gt;
              &lt;/Card&gt;
            &lt;/Col&gt;
```

```
            &lt;Col span={6}&gt;
              &lt;Card&gt;
                &lt;Statistic title="用户总数" value={stats ? stats.userTotal : 0} /&gt;
              &lt;/Card&gt;
            &lt;/Col&gt;
            &lt;Col span={6}&gt;
              &lt;Card&gt;
                &lt;Statistic title="已完成任务" value={stats ? stats.doneTaskTotal : 0} /&gt;
              &lt;/Card&gt;
            &lt;/Col&gt;
          &lt;/Row&gt;
        &lt;/Content&gt;
      &lt;/Layout&gt;
    &lt;/Layout&gt;
  );
}
// ============ 文件：admin/src/pages/Login.jsx ============
import { useState } from 'react';
import { Form, Input, Button, message } from 'antd';
import { UserOutlined, LockOutlined } from '@ant-design/icons';
import { authApi } from '../api';
import { useNavigate } from 'react-router-dom';

export default function Login() {
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  const onFinish = async (values) =&gt; {
    setLoading(true);
    try {
      const data = await authApi.login(values);
      localStorage.setItem('token', data.token);
      localStorage.setItem('userName', data.name);
      message.success('登录成功');
      navigate('/dashboard');
    } catch (e) {
      message.error(e.message || '登录失败');
    } finally {
      setLoading(false);
    }
  };

  return (
    &lt;div className="login-wrap"&gt;
      &lt;div className="login-box"&gt;
        &lt;h1 className="login-title"&gt;学习打卡管理后台&lt;/h1&gt;
        &lt;Form onFinish={onFinish} size="large"&gt;
          &lt;Form.Item name="account" rules={[{ required: true, message: '请输入账号' }]}&gt;
            &lt;Input prefix={&lt;UserOutlined /&gt;} placeholder="账号" /&gt;
          &lt;/Form.Item&gt;
```

```
          &lt;Form.Item name="password" rules={[{ required: true, message: '请输入密码' }]}&gt;
            &lt;Input.Password prefix={&lt;LockOutlined /&gt;} placeholder="密码" /&gt;
          &lt;/Form.Item&gt;
          &lt;Form.Item&gt;
            &lt;Button type="primary" htmlType="submit" block loading={loading}&gt;
              登录
            &lt;/Button&gt;
          &lt;/Form.Item&gt;
        &lt;/Form&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
// ============ 文件：admin/src/pages/StatsPage.jsx ============
import { useEffect, useState } from 'react';
import { Card, Row, Col, Table } from 'antd';
import { taskApi } from '../api';

export default function StatsPage() {
  const [rows, setRows] = useState([]);
  const [loading, setLoading] = useState(false);

  useEffect(() =&gt; {
    setLoading(true);
    taskApi
      .list({ pageSize: 100 })
      .then((data) =&gt; setRows(data.list))
      .finally(() =&gt; setLoading(false));
  }, []);

  const bySubject = {};
  rows.forEach((r) =&gt; {
    const key = r.subject || '未分类';
    bySubject[key] = (bySubject[key] || 0) + 1;
  });

  const subjectRows = Object.entries(bySubject).map(([subject, count]) =&gt; ({
    subject,
    count
  }));

  return (
    &lt;div style={{ padding: 16 }}&gt;
      &lt;Row gutter={16}&gt;
        &lt;Col span={12}&gt;
          &lt;Card title="任务科目分布" style={{ marginBottom: 16 }}&gt;
            &lt;Table
              rowKey="subject"
              dataSource={subjectRows}
              loading={loading}
```

```
              pagination={false}
              columns={[
                { title: '科目', dataIndex: 'subject' },
                { title: '任务数', dataIndex: 'count' }
              ]}
            /&gt;
          &lt;/Card&gt;
        &lt;/Col&gt;
        &lt;Col span={12}&gt;
          &lt;Card title="最近任务"&gt;
            &lt;Table
              rowKey="id"
              dataSource={rows.slice(0, 20)}
              loading={loading}
              pagination={false}
              columns={[
                { title: '标题', dataIndex: 'title' },
                { title: '状态', dataIndex: 'status' },
                { title: '创建时间', dataIndex: 'created_at' }
              ]}
            /&gt;
          &lt;/Card&gt;
        &lt;/Col&gt;
      &lt;/Row&gt;
    &lt;/div&gt;
  );
}
// ============ 文件：admin/src/pages/TaskManage.jsx ============
import { useEffect, useState } from 'react';
import { Table, Tag, Button, Modal, Form, Input, Select, message } from 'antd';
import { taskApi } from '../api';

export default function TaskManage() {
  const [list, setList] = useState([]);
  const [loading, setLoading] = useState(false);
  const [editing, setEditing] = useState(null);
  const [modalOpen, setModalOpen] = useState(false);
  const [form] = Form.useForm();
  const [pager, setPager] = useState({ pageNo: 1, pageSize: 20, total: 0 });

  const load = async () =&gt; {
    setLoading(true);
    try {
      const data = await taskApi.list(pager);
      setList(data.list);
      setPager((p) =&gt; ({ ...p, total: data.total }));
    } finally {
      setLoading(false);
    }
  };
```

```
useEffect(() =&gt; {
  load();
}, [pager.pageNo]);

const statusColor = { todo: 'default', doing: 'processing', done: 'success' };

const openCreate = () =&gt; {
  setEditing(null);
  form.resetFields();
  setModalOpen(true);
};

const openEdit = (row) =&gt; {
  setEditing(row);
  form.setFieldsValue(row);
  setModalOpen(true);
};

const submit = async () =&gt; {
  const values = await form.validateFields();
  try {
    if (editing) {
      await taskApi.update(editing.id, values);
    } else {
      await taskApi.create(values);
    }
    message.success('保存成功');
    setModalOpen(false);
    load();
  } catch (e) {
    message.error(e.message);
  }
};

const remove = (row) =&gt; {
  Modal.confirm({
    title: '确认删除该任务？',
    onOk: async () =&gt; {
      await taskApi.remove(row.id);
      message.success('删除成功');
      load();
    }
  });
};

const columns = [
  { title: 'ID', dataIndex: 'id', width: 80 },
  { title: '标题', dataIndex: 'title' },
  { title: '科目', dataIndex: 'subject' },
```

```
  {
    title: '状态',
    dataIndex: 'status',
    render: (s) =&gt; &lt;Tag color={statusColor[s]}&gt;{s}&lt;/Tag&gt;
  },
  { title: '创建人', dataIndex: 'creator_name' },
  {
    title: '操作',
    render: (_, row) =&gt; (
      &lt;Space&gt;
        &lt;Button size="small" onClick={() =&gt; openEdit(row)}&gt;
          编辑
        &lt;/Button&gt;
        &lt;Button size="small" danger onClick={() =&gt; remove(row)}&gt;
          删除
        &lt;/Button&gt;
      &lt;/Space&gt;
    )
  }
];

return (
  &lt;div style={{ padding: 16 }}&gt;
    &lt;Button type="primary" style={{ marginBottom: 16 }} onClick={openCreate}&gt;
      新建任务
    &lt;/Button&gt;
    &lt;Table
      rowKey="id"
      loading={loading}
      columns={columns}
      dataSource={list}
      pagination={{
        current: pager.pageNo,
        pageSize: pager.pageSize,
        total: pager.total,
        onChange: (pageNo) =&gt; setPager((p) =&gt; ({ ...p, pageNo }))
      }}
    /&gt;
    &lt;Modal
      title={editing ? '编辑任务' : '新建任务'}
      open={modalOpen}
      onOk={submit}
      onCancel={() =&gt; setModalOpen(false)}
      destroyOnClose
    &gt;
      &lt;Form form={form} labelCol={{ span: 5 }}&gt;
        &lt;Form.Item name="title" label="标题" rules={[{ required: true, message: '请输入标题' }]}&gt;
          &lt;Input /&gt;
        &lt;/Form.Item&gt;
        &lt;Form.Item name="subject" label="科目"&gt;
```

```
            &lt;Select
              options={['语文', '数学', '英语', '阅读', '运动', '其他'].map((s) =&gt; ({
                label: s,
                value: s
              }))}
            /&gt;
          &lt;/Form.Item&gt;
          &lt;Form.Item name="description" label="描述"&gt;
            &lt;Input.TextArea rows={4} /&gt;
          &lt;/Form.Item&gt;
          &lt;Form.Item name="score" label="分值"&gt;
            &lt;Input type="number" /&gt;
          &lt;/Form.Item&gt;
        &lt;/Form&gt;
      &lt;/Modal&gt;
    &lt;/div&gt;
  );
}
// ============ 文件：admin/src/pages/UserManage.jsx ============
import { useEffect, useState } from 'react';
import { Table, Input, Space, Button, Switch, message } from 'antd';
import { userApi } from '../api';

export default function UserManage() {
  const [list, setList] = useState([]);
  const [loading, setLoading] = useState(false);
  const [keyword, setKeyword] = useState('');
  const [pager, setPager] = useState({ pageNo: 1, pageSize: 20, total: 0 });

  const load = async () =&gt; {
    setLoading(true);
    try {
      const data = await userApi.list({ keyword, ...pager });
      setList(data.list);
      setPager((p) =&gt; ({ ...p, total: data.total }));
    } finally {
      setLoading(false);
    }
  };

  useEffect(() =&gt; {
    load();
  }, [pager.pageNo, pager.pageSize]);

  const toggleStatus = async (row, checked) =&gt; {
    try {
      await userApi.setStatus({ userId: row.id, status: checked ? 1 : 0 });
      message.success('操作成功');
      load();
    } catch (e) {
```

```
      message.error(e.message);
    }
  };

  const columns = [
    { title: 'ID', dataIndex: 'id' },
    { title: '账号', dataIndex: 'account' },
    { title: '姓名', dataIndex: 'name' },
    { title: '角色', dataIndex: 'role' },
    {
      title: '状态',
      dataIndex: 'status',
      render: (_, row) =&gt; (
        &lt;Switch checked={row.status === 1} onChange={(v) =&gt; toggleStatus(row, v)} /&gt;
      )
    },
    { title: '注册时间', dataIndex: 'created_at' }
  ];

  return (
    &lt;div style={{ padding: 16 }}&gt;
      &lt;Space style={{ marginBottom: 16 }}&gt;
        &lt;Input.Search
          placeholder="搜索账号/姓名"
          allowClear
          onSearch={(v) =&gt; {
            setKeyword(v);
            setPager((p) =&gt; ({ ...p, pageNo: 1 }));
          }}
          style={{ width: 260 }}
        /&gt;
        &lt;Button type="primary" onClick={load}&gt;
          刷新
        &lt;/Button&gt;
      &lt;/Space&gt;
      &lt;Table
        rowKey="id"
        loading={loading}
        columns={columns}
        dataSource={list}
        pagination={{
          current: pager.pageNo,
          pageSize: pager.pageSize,
          total: pager.total,
          onChange: (pageNo, pageSize) =&gt; setPager((p) =&gt; ({ ...p, pageNo, pageSize }))
        }}
      /&gt;
    &lt;/div&gt;
  );
}
```